

SISTEMA idUPSF

Documentação Final de Arquitetura e Implementação

Padrões e Princípios de Projeto de Software
Junho de 2026

1. Aplicação de Princípios e Padrões

1.1 Princípios SOLID

Os princípios SOLID orientaram a estrutura do sistema idUPSF desde sua concepção, promovendo código manutenível, extensível e testável. A seguir, cada princípio é descrito com exemplos concretos extraídos da implementação.

S - Single Responsibility Principle (Princípio da Responsabilidade Única)

Cada classe possui uma única razão para mudar. O sistema é estruturado em camadas com responsabilidades bem delimitadas:

- Controllers: recebem e roteiam requisições HTTP
- Services: orquestram casos de uso e regras de aplicação
- Entidades de domínio: concentram regras de negócio
- Repositories: abstraem o acesso ao banco
- Mappers (MapStruct): convertem entre entidade e DTO

Exemplo concreto - Historico.calcularCR():

A lógica de cálculo do Coeficiente de Rendimento vive na própria entidade, não no serviço. HistoricoService.atualizarCoeficiente() delega ao modelo, respeitando SRP:

```
// Historico.java
public float calcularCR() {
    if (this.listaDisciplinas == null || this.listaDisciplinas.isEmpty()) return 0f;

    float somaNotasPonderadas = 0;
    int somaCargaHoraria = 0;
    for (DisciplinaCursada disciplina : this.listaDisciplinas) {
        if (disciplina.getNota() >= 6.0)
            somaNotasPonderadas += disciplina.getNota() *
disciplina.getCargaHoraria();
        else
            somaNotasPonderadas += disciplina.getNotaVS() *
disciplina.getCargaHoraria();
        somaCargaHoraria += disciplina.getCargaHoraria();
    }
    this.coeficienteRend = somaNotasPonderadas / somaCargaHoraria;
    return this.coeficienteRend;
}
```

Violação identificada e corrigida: Antes da refatoração, a lógica de cálculo do CR estava duplicada em HistoricoService. Ela foi movida para a entidade Historico, que detém os dados necessários. O serviço passou a apenas invocar historico.calcularCR() e persistir o resultado.

O - Open/Closed Principle (Aberto para extensão, fechado para modificação)

O sistema de validação de matrícula é extensível sem modificar InscricaoService. Basta implementar a interface ValidacaoInscricao e anotar com @Component, o Spring injeta automaticamente a nova validação na lista:

```
// Antes (violação): toda nova regra exigia modificar InscricaoService
```

```
public void realizarInscricao(InscricaoCreate dto) {  
    // if (conflito de horário) ...  
    // if (sem vagas) ...  
    // if (nova regra) ... <- modificava o método  
}  
  
// Depois (conforme OCP): nova regra = nova classe, zero alteração no Service  
@Component  
public class ValidarVagasDisponiveis implements ValidacaoInscricao {  
    @Override  
    public void validar(Discente discente, List<Turma> turmasSelecionadas) { ...  
}  
}
```

O mesmo princípio se aplica a `GeradorDePDFService<T>`: novos tipos de documento estendem a classe abstrata sem alterar o algoritmo principal (`gerarPdf()`).

L - Liskov Substitution Principle

A hierarquia `Usuario` → `Discente` / `Docente` → `Coordenador` respeita LSP. `Discente` e `Docente` podem ser usados em qualquer contexto que espere `Usuario` sem quebrar contratos. A herança JOINED no JPA reflete essa hierarquia diretamente no banco de dados.

`HistoricoPDFService` e `RegularidadePDFService` são substituíveis por `GeradorDePDFService<T>` em qualquer ponto de injeção, nenhuma subclasse altera o comportamento esperado do método `gerarPdf()`.

I - Interface Segregation Principle

A interface `ValidacaoInscricao` expõe apenas um método, impedindo que implementações dependam de métodos desnecessários:

```
public interface ValidacaoInscricao {  
    void validar(Discente discente, List<Turma> turmasSelecionadas);  
}
```

Os repositórios JPA também seguem ISP, cada um herda de `JpaRepository` apenas os métodos necessários e declara apenas queries específicas ao seu contexto.

D - Dependency Inversion Principle

`InscricaoService` não depende de implementações concretas de validação. Depende da abstração `ValidacaoInscricao`, e o Spring resolve a lista de implementações em tempo de execução:

```
@Autowired  
private List<ValidacaoInscricao> validacoesInscricao;  
  
// No método:  
validacoesInscricao.forEach(v -> v.validar(discente, turmasSelecionadas));
```

Todos os serviços dependem de interfaces de repositório (`InscricaoRepository`, `TurmaRepository`), nunca de implementações JPA concretas.

1.2 Padrões GRASP

Os padrões GRASP (General Responsibility Assignment Software Patterns) orientaram a atribuição de responsabilidades entre as classes do sistema.

Especialista em Informação

A responsabilidade de processar uma informação é atribuída à classe que detém os dados necessários:

Operação	Classe Responsável	Justificativa
Calcular CR	Historico	Possui listaDisciplinas com notas e cargas horárias
Determinar aprovação/reprovação	DisciplinaCursada	Possui nota, notaVS e frequencia
Detectar conflito de horário	Horario	Possui diasDaSemana, horarioInicio e horarioFim
Gerenciar pré-requisitos	Disciplina	Possui a própria lista preRequisitos

```
// DisciplinaCursada.java - Especialista em Informação
public void calcularStatusFinal() {
    if (this.frequencia && this.statusFinal != StatusFinal.TRANCADO) {
        if (this.nota >= 6.0)          this.statusFinal = StatusFinal.APROVADO;
        else if (this.notaVS >= 6.0) this.statusFinal = StatusFinal.APROVADO;
        else                        this.statusFinal = StatusFinal.REPROVADO;
    } else {
        this.statusFinal = StatusFinal.REPROVADO;
    }
}
```

Criador

InscricaoService cria instâncias de Inscricao porque detém todas as informações necessárias (Turma e Discente). Da mesma forma, DiscenteService cria Discente e o Historico associado na mesma operação de cadastro.

Controller

Os @RestController atuam como Controllers GRASP: recebem eventos do sistema (requisições HTTP), não contêm lógica de negócio, e delegam para os Services apropriados.

Baixo Acoplamento

- Serviços comunicam-se via interfaces, nunca por acesso direto a outros serviços desnecessariamente
- DTOs isolam a camada de apresentação do domínio
- ValidacaoInscricao desacopla o serviço das regras de validação específicas

Alta Coesão

- Horário concentra apenas dados e comportamento relacionados a horários
- GeradorDePDFService<T> concentra apenas responsabilidades de geração de documentos PDF
- TurmaSpec concentra apenas a lógica de especificação para filtros dinâmicos de turmas

2. Padrões GoF Implementados

2.1 Strategy - Validação de Matrícula

Contexto de uso:

O processo de inscrição em turmas exige múltiplas validações independentes entre si (conflito de horário, disponibilidade de vagas, etc.).

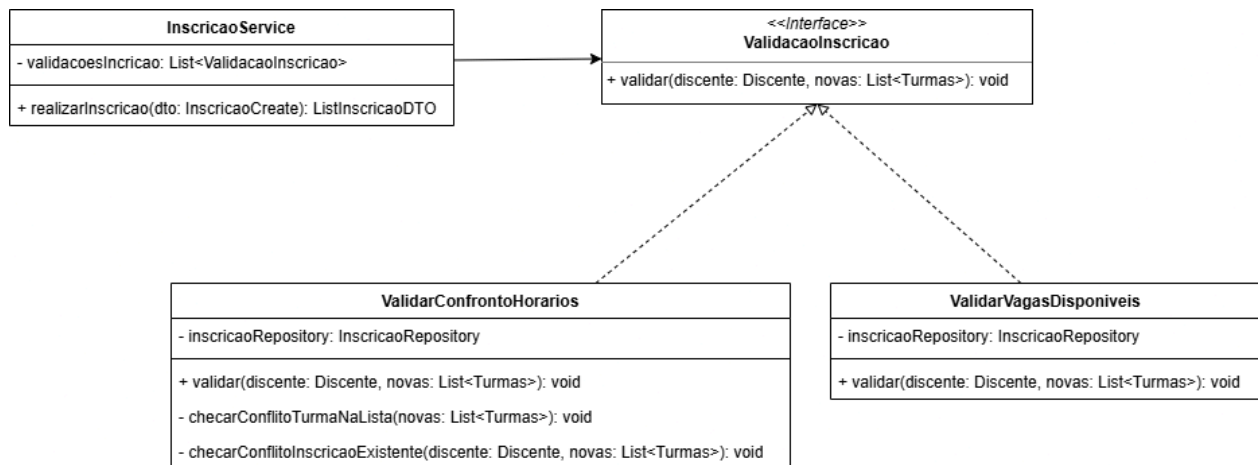
Problema que resolve:

Sem o padrão, todas as regras de validação estariam dentro de InscricaoService em blocos if/else sequenciais. Cada nova regra exigiria modificar o método realizarInscricao(), violando OCP e tornando o código difícil de testar e estender.

Justificativa da escolha:

Strategy permite encapsular cada algoritmo de validação em uma classe separada, injetar qualquer número delas via lista, e adicionar novas regras sem tocar no código existente. O Spring gerencia a injeção automaticamente pela anotação @Component.

Diagrama de classes - Padrão Strategy:



```

@Autowired
private List<ValidacaoInscricao> validacoesInscricao;

public List<InscricaoDTO> realizarInscricao(InscricaoCreate dto) {
    // ... buscar discente e turmas ...
    validacoesInscricao.forEach(v -> v.validar(discente, turmasSelecionadas));
    return inscricaoRepository.saveAll(inscricoes)
        .stream().map(inscricaoMapper::toDTO).toList();
}
  
```

2.2 Template Method - Geração de Documentos PDF

Contexto de uso:

O sistema gera dois tipos de documento PDF: Histórico Escolar e Declaração de Regularidade de Matrícula. Ambos compartilham a mesma estrutura de geração (abrir documento, inserir cabeçalho padrão com logo, construir conteúdo, fechar), mas o conteúdo específico de cada documento é diferente.

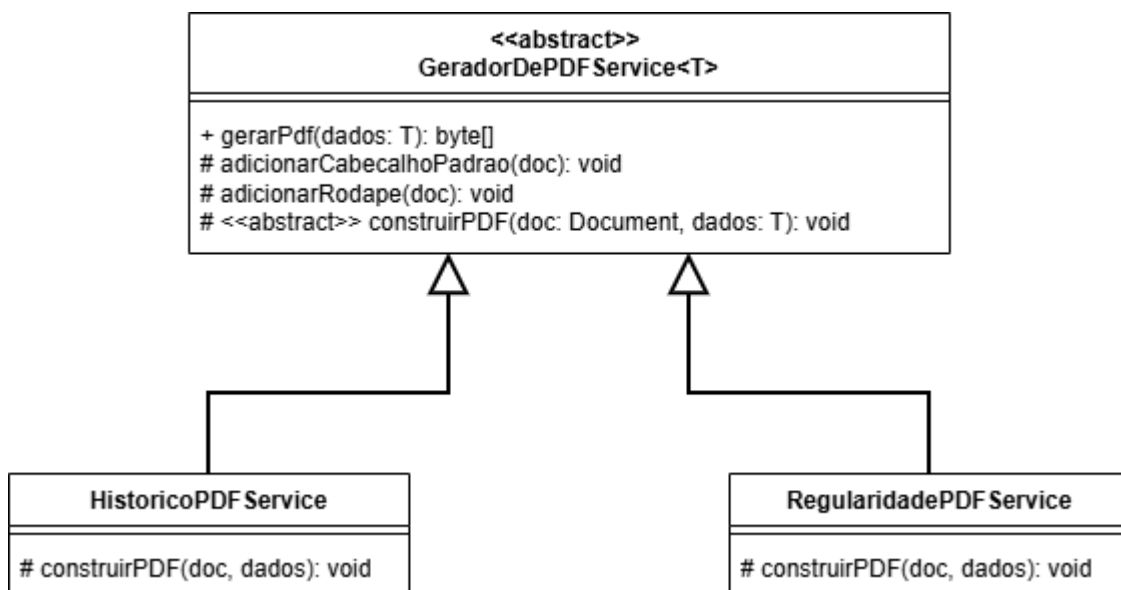
Problema que resolve:

Sem o padrão, o código de inicialização do documento, inserção de cabeçalho e finalização estaria duplicado em cada serviço de geração. Qualquer mudança no cabeçalho padrão exigiria replicação em todos os geradores.

Justificativa da escolha:

Template Method define o esqueleto do algoritmo na classe abstrata e delega apenas as etapas variáveis para as subclasses. O método gerarPdf() é o template imutável. Apenas construirPDF() é abstrato e varia por subtipo.

Diagrama de classes - Padrão Template Method:



Fluxo do template (gerarPdf):

```
gerarPdf(dados)
|-- document.open()
|-- adicionarCabecalhoPadrao()    <- fixo (logo institucional)
|-- construirPDF()                <- abstrato, implementado pela subclasse
`-- document.close()
```

```
// GeradorDePDFService.java - AbstractClass
public abstract class GeradorDePDFService<T> {
    public byte[] gerarPdf(T dados) {
        Document document = new Document();
```

```
document.open();  
adicionarCabecalhoPadrao(document);  
construirPDF(document, dados); // hook  
document.close();  
return baos.toByteArray();  
}  
protected abstract void construirPDF(Document document, T dados);  
}
```

2.3 Facade

Contexto de uso:

O processo de matrícula do idUPSF envolve a orquestração de múltiplos subsistemas independente, verificação de período letivo ativo, busca de disciplinas aprovadas no histórico, carregamento de turmas disponíveis, execução das estratégias de validação e persistência das inscrições, todos acessados diretamente pelo `InscricaoService` e expostos indiretamente ao `InscricaoController`.

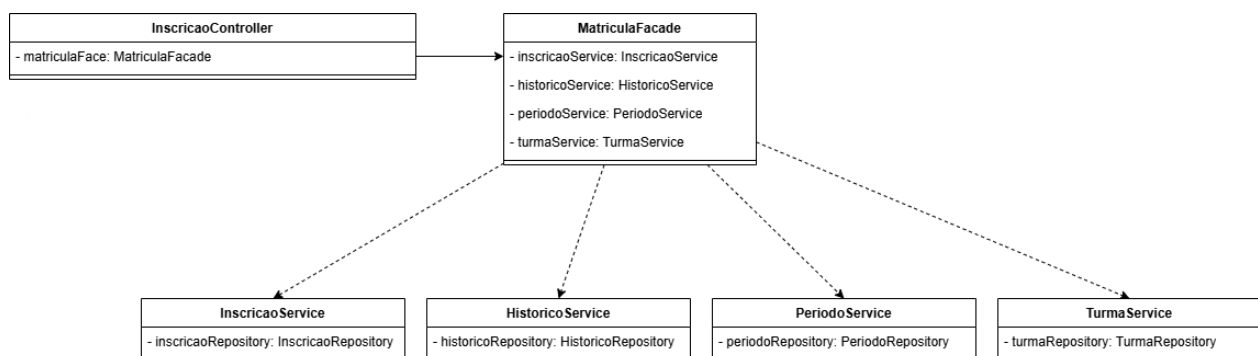
Problema que resolve:

Sem o padrão, o `InscricaoService` acumulava duas responsabilidades distintas: orquestrar a colaboração entre `PeriodoRepository`, `HistoricoService`, `TurmaRepository` e as estratégias de `ValidacaoInscricao`, além de executar a lógica própria de inscrição, violando o SRP; o controlador, por sua vez, dependia indiretamente de toda essa complexidade interna, tornando o sistema difícil de testar, estender e manter.

Justificativa da escolha:

O `MatriculaFacade` passa a ser o único ponto de entrada para o processo de matrícula, o controlador chama um único método (`realizarMatricula()`, `listarTurmasDisponiveis()`) sem conhecer a existência de `HistoricoService`, `PeriodoService` ou das estratégias de validação; cada subsistema retoma sua responsabilidade restrita; e a complexidade da orquestração fica centralizada em uma única classe, tornando qualquer alteração no fluxo de matrícula localizada e sem impacto nos subsistemas individuais nem no controlador.

Diagrama de classes - Padrão Facade:



2.4 State

Contexto de uso:

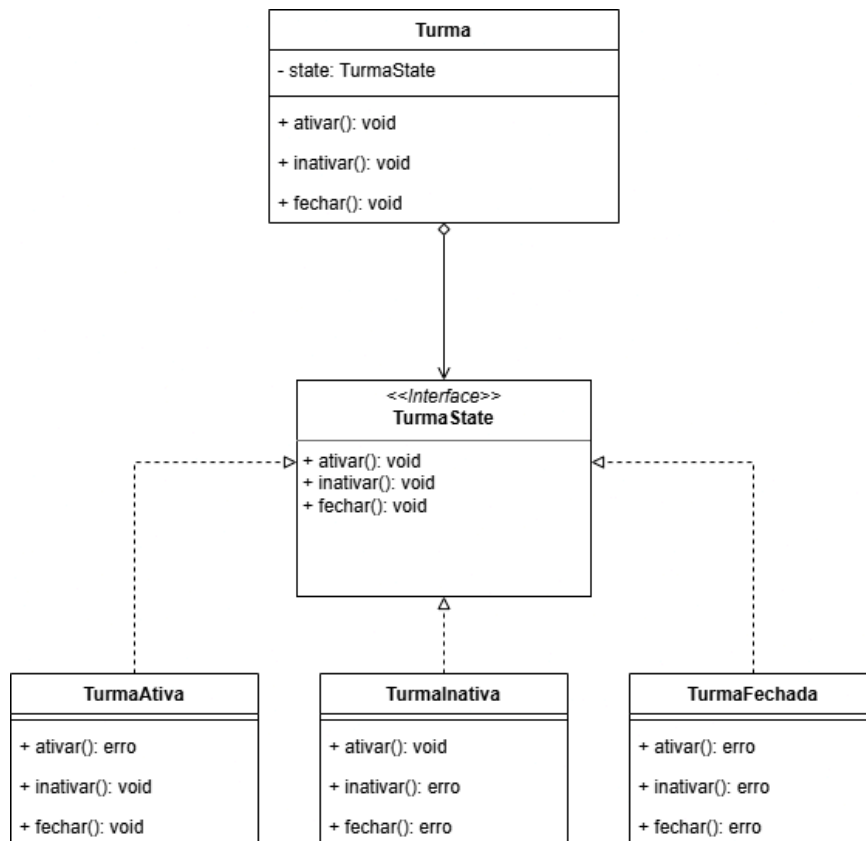
A entidade Turma possui três estados distintos (ATIVA, INATIVA, FECHADA) que determinam quais operações são permitidas, se inscrições podem ser realizadas, se um docente pode ser atribuído e quais transições de estado são válidas, e a entidade DisciplinaCursada segue lógica semelhante com os estados AGUARDADO, APROVADO, REPROVADO e TRANCADO.

Problema que resolve:

Sem o padrão, essas regras estavam espalhadas por TurmaService e InscricaoService na forma de verificações condicionais explícitas (if (turma.getStatus() != ATIVA) throw ...), tornando o código frágil: adicionar um novo estado ou alterar uma regra de transição exigia localizar e modificar múltiplos pontos do sistema, com risco real de inconsistências silenciosas como fechar uma turma já fechada sem que o chamador fosse obrigado a tratar o erro.

Justificativa da escolha:

O padrão State encapsula cada comportamento dentro do próprio objeto de estado concreto (TurmaAtiva, TurmaInativa, TurmaFechada), de modo que Turma delega as operações ativar(), inativar(), fechar() e validarInscricao() para o estado vigente, que lança RegraNegocioException sempre que a transição é inválida, tornando erros de lógica impossíveis de ignorar pelo chamador, e os serviços passam a expressar intenção (turma.ativar()) em vez de manipular status diretamente, eliminando toda a lógica condicional duplicada e concentrando as regras de negócio onde os dados residem.

Diagrama de classes - Padrão State:

2.5 Factory Method

Contexto de uso:

O sistema idUPSF possui três fluxos de cadastro de usuários: Discente, Docente e Coordenador, que compartilham uma sequência de etapas quase idêntica: validar duplicatas de CPF e e-mail, mapear o DTO para a entidade, gerar matrícula institucional, gerar e-mail institucional, definir status como ATIVO e data de ingresso, e persistir no repositório; o que varia entre eles é exclusivamente o tipo de objeto criado e as regras específicas de cada criação, como o formato da matrícula e as associações com Curso, Departamento ou Histórico.

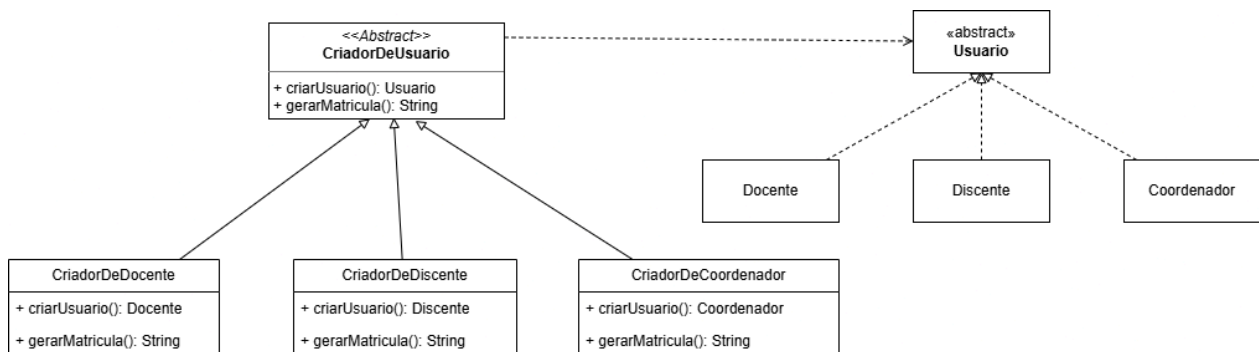
Problema que resolve:

Sem o padrão, essa sequência estava duplicada inline em DiscenteService, DocenteService e CoordenadorService, cada um com cerca de 25 linhas de lógica de criação repetida, além de uma classe utilitária IdentificacaoUsuarioUtil deslocada que concentrava funções de geração de matrícula e e-mail sem pertencer claramente a nenhum domínio; qualquer alteração no algoritmo de cadastro, como adicionar uma nova etapa obrigatória, precisava ser replicada manualmente nos três serviços.

Justificativa da escolha:

O Factory Method permite que CriadorDeUsuario<T, C> defina o algoritmo fixo de registro no método concreto registrar() e delegue apenas a etapa de criação do objeto para o factory method abstrato criarUsuario(), que cada subclasse (CriadorDeDiscente, CriadorDeDocente, CriadorDeCoordenador) implementa com suas particularidades; os serviços passam a invocar um único método (criador.registrar(dto)), a lógica comum fica centralizada na classe base, e adicionar um novo tipo de usuário exige apenas criar uma nova subclasse sem tocar em nenhum serviço existente.

Diagrama de classes - Padrão Factory Method:

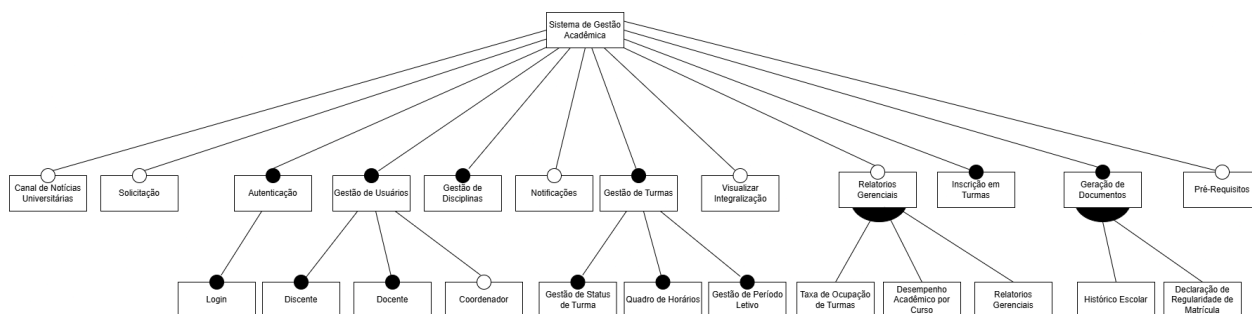


3. Linha de Produto de Software (LPS)

O sistema idUPSF pode ser concebido como uma LPS voltada a instituições de ensino superior com diferentes perfis de complexidade e requisitos operacionais. O modelo de características a seguir foi derivado da implementação do projeto, identificando as características mandatórias (círculo preenchido), opcionais (círculo vazio) e os grupos de subcaracterísticas de cada módulo.

3.1 Modelo de Características

Característica	Tipo	Subcaracterísticas
Autenticação	Mandatória	-
Gestão de Usuários	Mandatória	Discente (mandatório), Docente (mandatório), Coordenador (opcional)
Gestão de Disciplinas	Mandatória	-
Gestão de Turmas	Mandatória	Gestão de Status de Turma (mandatório), Quadro de Horários (mandatório), Gestão de Período Letivo (mandatório)
Inscrição em Turmas	Mandatória	-
Geração de Documentos	Mandatória	Histórico Escolar (mandatório), Declaração de Regularidade de Matrícula (mandatório)
Relatórios Gerenciais	Opcional	Taxa de Ocupação de Turmas (mandatório no grupo), Desempenho Acadêmico por Curso (mandatório no grupo), Relatórios Gerenciais gerais (mandatório no grupo)
Visualizar Integralização	Opcional	-
Notificações	Opcional	-
Pré-Requisitos	Opcional	-
Canal de Notícias Universitárias	Opcional	-
Solicitação	Opcional	-



4. Descrição da Implementação

4.1 Tecnologias e Frameworks

Tecnologia	Papel na Solução
Spring Boot 4.0.6	Framework principal do backend; gerencia ciclo de vida dos beans, injeção de dependências e exposição de endpoints REST
Spring Data JPA / Hibernate	Framework de persistência; abstrai o acesso ao banco via repositórios e gerencia o mapeamento objeto-relacional e herança JOINED
Spring Security	Camada de autenticação; intercepta requisições e autentica usuários via DaoAuthenticationProvider + UsuarioDetailsService
MapStruct	Geração de mappers objeto-objeto em tempo de compilação; converte entidades em DTOs sem lógica manual
OpenPDF	Geração de documentos PDF; utilizado na implementação do padrão Template Method
Lombok	Redução de boilerplate (@Getter, @Setter, @RequiredArgsConstructor, @NoArgsConstructor)
MySQL	Banco de dados relacional para persistência de todos os dados do sistema
Next.js 16 / React 19	Framework do front-end; renderização das telas e roteamento client-side
TypeScript	Linguagem utilizada no frontend
NextAuth	Gerenciamento de sessão e autenticação no lado do cliente

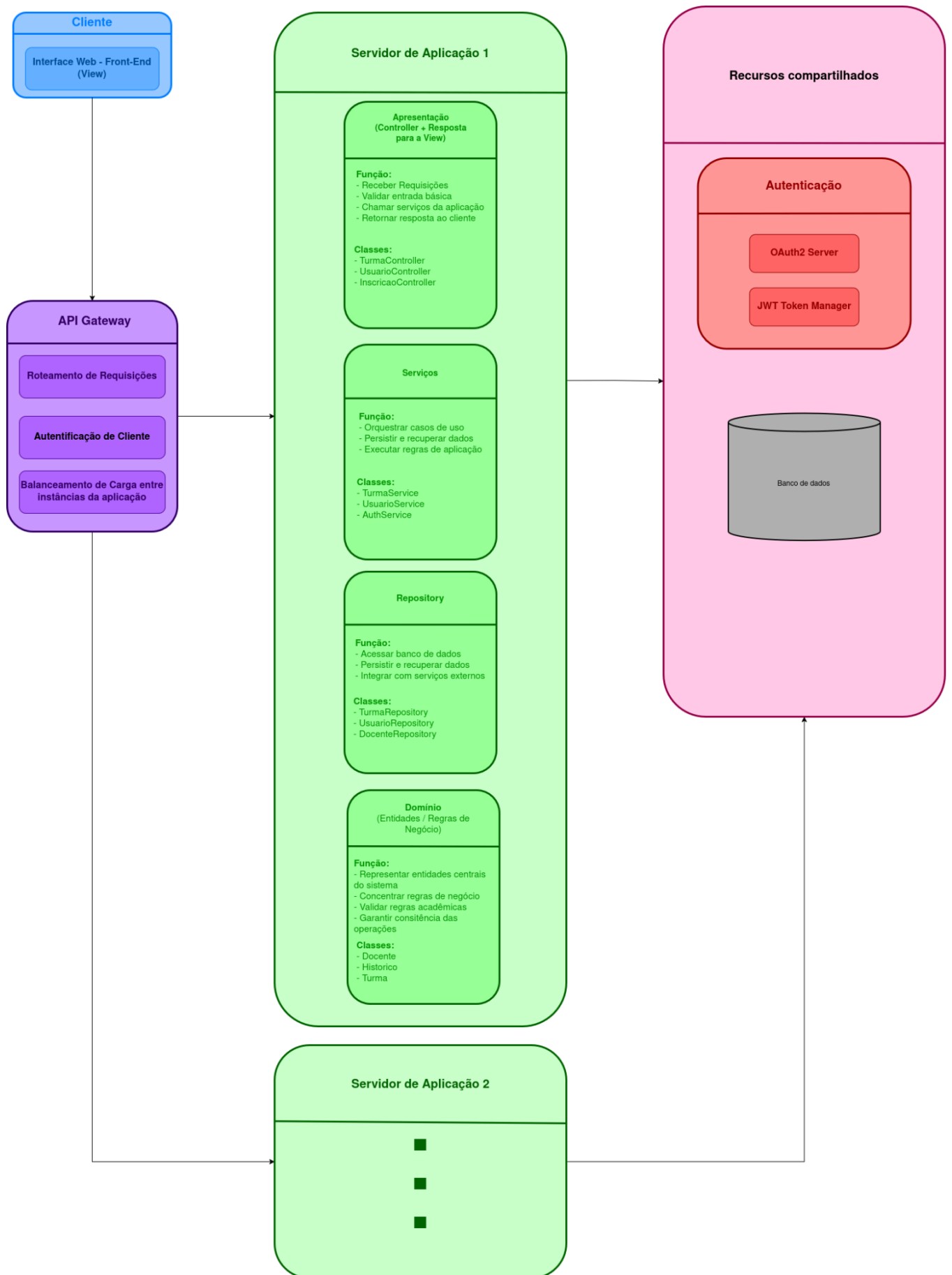
4.2 Padrões de Projeto Empregados nos Frameworks

- Repository Pattern - implementado via JpaRepository; cada entidade possui seu próprio repositório que encapsula o acesso ao banco
- DTO Pattern - objetos de transferência de dados (pacotes dto, create, update) isolam a API do domínio
- Specification Pattern - TurmaSpec utiliza JpaSpecificationExecutor para construção de queries dinâmicas sem SQL concatenado

4.3 Arquitetura Geral

A solução segue uma arquitetura em camadas com separação clara entre frontend, API Gateway, servidor de aplicação e recursos compartilhados. A arquitetura final não apresenta diferenças estruturais em relação à primeira entrega, as mesmas quatro camadas internas foram mantidas:

- Apresentação: Next.js (frontend) com componentes React tipados em TypeScript
- Serviços: Spring Boot controllers e services com injeção de dependências
- Domínio: Entidades JPA com regras de negócio encapsuladas
- Infraestrutura: Repositórios JPA/Hibernate, banco MySQL



5. Implementação do Sistema

5.1 CRUDs Implementados

Para cada membro da equipe foi implementado pelo menos CRUD completo (criar, buscar, atualizar, remover):

Entidade	Operações	Controller
Discente	Criar, buscar, atualizar, remover	DiscenteController
Docente	Criar, buscar, atualizar, remover	DocenteController
Coordenador	Criar, buscar, atualizar, remover	CoordenadorController
Turma	Criar, buscar, atualizar, remover	TurmaController
Disciplina	Criar, buscar, atualizar, remover (+ pré-requisitos)	DisciplinaController
Curriculo	Criar, buscar, atualizar, remover	CurriculoController
Curso	Criar, buscar, atualizar, remover	CursoController
Periodo	Criar, buscar, atualizar datas, adicionar/remover turmas	PeriodoController
Inscrição	Criar, buscar, remover inscrições	InscricaoController

5.2 Casos de Uso Transacionais

Caso de Uso 1: Realizar Inscrição em Turmas (POST /api/inscricoes)

Operação transacional (@Transactional) que garante que o aluno só é inscrito se todas as etapas forem concluídas com sucesso. Em caso de falha, toda a operação é desfeita:

- Verificar se há período de inscrição aberto
- Buscar o Discente e as Turmas selecionadas
- Executar todas as estratégias de validação (ValidarConfrontoHorarios, ValidarVagasDisponiveis)
- Persistir todas as Inscricoes de uma vez (inscricaoRepository.saveAll())

Nota: Se qualquer validação lançar RegraNegocioException, nenhuma inscrição é salva.

Caso de Uso 2: Atribuir Notas e Frequências (PATCH /api/turmas/{turmaId}/notas)

Operação transacional que valida autoria e integridade antes de persistir qualquer alteração:

- Verificar se a turma existe
- Verificar se o docente informado é o responsável pela turma
- Verificar se a turma não está com status FECHADA
- Para cada atualização, verificar se a inscrição pertence à turma informada
- Persistir as notas e frequências

Nota: Se qualquer inscrição não pertencer à turma, nenhuma atualização é confirmada.

5.3 Caso de Uso de Listagem (Relatório)

Listar Turmas Disponíveis para Matrícula (GET /api/inscricoes/turmas-disponiveis/{discenteId})

Gera uma listagem filtrada das turmas nas quais o aluno pode se matricular, respeitando as regras acadêmicas:

- Buscar todas as disciplinas já aprovadas no histórico do discente
- Buscar todas as turmas com status ATIVA
- Filtrar turmas cujos pré-requisitos estão todos dentro do conjunto de disciplinas aprovadas
- Retornar a lista resultante como DTOs

```
public List<TurmaDTO> listarTurmasDisponiveis(Long discenteId) {  
    Set<Disciplina> concluidas = new HashSet<>(  
        historicoService.buscarDisciplinasEntitiesAprovadas(discenteId)  
    );  
    List<Turma> turmasAtivas =  
        turmaRepository.buscarTurmasAtivasComRequisitos();  
  
    return turmasAtivas.stream()  
        .filter(t ->  
            concluidas.containsAll(t.getDisciplina().getPreRequisitos()))  
        .map(turmaMapper::toTurmaDTO)  
        .toList();  
}
```

Gerar PDF do Histórico Escolar (GET /api/documentos/historico/{discenteId})

Gera e retorna um documento PDF contendo o histórico acadêmico completo do aluno. O documento inclui dados cadastrais do discente, a lista de todas as disciplinas cursadas com suas respectivas notas, notas de verificação suplementar, carga horária, frequência e status final (aprovado, reprovado ou trancado), além do Coeficiente de Rendimento (CR) calculado ao final.

A geração é realizada por `HistoricoPDFService`, que estende a classe abstrata `GeradorDePDFService<RelatorioHistoricoDTO>` e implementa o método `construirPDF()`, o único passo variável do algoritmo definido pelo padrão `Template Method`. O método `gerarPdf()` da superclasse orquestra a sequência fixa (abertura do documento, inserção do cabeçalho institucional padrão com logo, delegação para `construirPDF()` e fechamento), enquanto `HistoricoPDFService` concentra exclusivamente a montagem do conteúdo específico do histórico: a tabela de disciplinas e o CR consolidado.

Fluxo de execução:

- Buscar o `RelatorioHistoricoDTO` do discente via `HistoricoService`
- Invocar `HistoricoPDFService.gerarPdf(dados)`, que aciona o template da superclasse
- `adicionarCabecalhoPadrao()` insere logo e dados institucionais fixos
- `construirPDF()` monta a tabela de disciplinas cursadas, notas e o CR final
- O documento é fechado e retornado como array de bytes (`application/pdf`)

```
@Service
public class HistoricoPDFService extends
GeradorDePDFService<RelatorioHistoricoDTO> {

    @Override
    protected void construirPDF(Document document, RelatorioHistoricoDTO dados)
throws Exception {}
}
```